

Lab 17: Math and Random

- Due Friday by 11:59pm
- Points 5
- Submitting an external tool
- Available Jul 8 at 6pm - Jul 10 at 11:59pm

Learning Objectives

- Practice writing tests for function.

Completion Requirements

- Submit Lab17.py

Question 1: The Chaos Playlist Generator

Write a function called `generate_playlist(song_pool, target_duration)`.

- `song_pool` is a dictionary where the keys are song titles (strings) and the values are their durations in seconds (integers).
- `target_duration` is an integer representing the maximum allowed length of the playlist in seconds.

Your function must dynamically build a playlist (a list of song titles) by randomly selecting songs from the pool. However, it must follow these strict rules:

1. The total duration of the playlist **cannot exceed** the `target_duration`.
2. The same song cannot be played twice in the playlist.

-  3. The function should keep trying to add random songs until adding any un-played song from the pool would push the total time over the `target_duration`.

You may consider creating a pool of songs that are eligible for selection before doing a selection as a helpful step. The function must return a tuple containing the final playlist (list) and the total actual duration of the playlist (integer).

Example 1: Normal case

```
songs = {"Song A": 180, "Song B": 240, "Song C": 120}
result_playlist, total_time = generate_playlist(songs, 500)
```

Possible valid answers:

```
# ([ 'Song A', 'Song B'], 420) or ([ 'Song B', 'Song C'], 360)
# Note: [ 'Song C', 'Song A', 'Song C'] is INVALID because 'Song C' appears twice.
```

Example 2: Empty case

```
songs = {"Epic Track": 600, "Long Symphony": 1200}
print(generate_playlist(songs, 300))

# Expected Output:
# ([], 0)
```

Example 3: All songs case

```
songs = {"Epic Track": 600, "Long Symphony": 1200}
print(generate_playlist(songs, 3000))

# Expected Output:
# ([ "Epic Track", "Long Symphony"], 1800)
```

Validation Rules for Grading: For each test, we will run the function at least 10 times because the output is randomized and check the rules for output in each case. We will verify that that function produces different outputs with the same input to check that function is truly random. The empty case will only be tested once.

Question 2: Projectile Trajectory Analyzer

Write a function called `analyze_trajectories(v_initial)`.

- `v_initial` is a float or integer representing the initial launch velocity of the projectile in meters per second (m/s).

Your function must calculate the horizontal range and maximum height across a specific sequence of angles. However, it must follow these strict rules:

- The calculations must use a fixed gravitational constant

$$g = 9.81 \text{ m/s}^2$$

- If the `v_initial` is less than or equal to 0, the function must immediately return an empty dictionary `{}`.
- The function must compute both the horizontal range (**d**) and maximum height (**h**) for angles from 15° to 75° inclusive, stepping by 5° at a time (i.e., 15, 20, 25, ..., 75).

The mathematical formulas are:

$$d = \frac{v_{\text{initial}}^2 \cdot \sin(2\theta)}{g}$$

and

$$h = \frac{v_{\text{initial}}^2 \cdot \sin^2(\theta)}{2g}$$

where θ is the launch angle converted from degrees to radians using the formula

$$\theta_{\text{radians}} = \theta_{\text{degrees}} \cdot \frac{\pi}{180}$$

The function must return a dictionary where the keys are the angles as integers, and the values are tuples containing the calculated distance and height rounded to 2 decimal places: `{angle: (distance, height)}`.

Example 1: Normal case

```
print(analyze_trajectories(10))
```

Expected Output:
 # {15: (5.1, 0.34), 20: (6.55, 0.59), 25: (7.81, 0.91), 30: (8.83, 1.27), 35: (9.58, 1.68), 40: (10.04, 2.11), 45: (10.19, 2.55), 50: (10.04, 2.99), 55: (9.58, 3.42), 60: (8.83, 3.82), 65: (7.81, 4.18), 70: (6.55, 4.5), 75: (5.1, 4.76)}

Example 2: Invalid case

```
print(analyze_trajectories(-12))
```

Expected Output:
 # {}

Example 3: High velocity case

```
result = analyze_trajectories(50)
print(result[45])
```

Expected Output:
 # (254.84, 63.71)

Validation Rules for Grading: We will verify that the function correctly handles edge cases by passing zero or negative velocities, ensuring it returns an empty dictionary. For valid velocities, we will verify the dictionary contains exactly 13 elements and strictly evaluate the precision and mathematical correctness of values at the boundaries (15° and 75°) as well as the maximum range angle (45°).

Question 3: Monte Carlo Sphere Volume Estimator

The **Monte Carlo method** is a mathematical technique that uses random sampling to solve problems that might be too complex or difficult to calculate using standard algebra or calculus.

Write a function called `monte_carlo_sphere_volume(radius, num_samples)`.

- `radius` is a float or integer representing the radius of the sphere.

- `num_samples` is an integer representing the total number of random coordinate points to generate for the simulation.

Your function must estimate the volume of a 3D sphere using a Monte Carlo simulation and compare it against the geometric formula. However, it must follow these strict rules:

1. The total volume of the bounding cube must be computed as

$$V_{\text{cube}} = (2 \cdot \text{radius})^3$$

2. If either `radius` or `num_samples` is less than or equal to 0, the function must immediately return `(0.0, 0.0)`.

3. The simulation must generate `num_samples` random points, each with floating-point **x**, **y**, and **z** coordinates between `-radius` and `+radius` inclusive. A point is inside the sphere if its 3D distance from the origin satisfies

$$\sqrt{x^2 + y^2 + z^2} \leq \text{radius}$$

The function must return a tuple containing the `(estimated_volume, actual_volume)`, both rounded to 4 decimal places. The actual volume must be calculated using the mathematical formula:

$$V_{\text{actual}} = \frac{4}{3}\pi \cdot \text{radius}^3$$

Example 1: Normal case

```
print(monte_carlo_sphere_volume(1, 100000))

# Possible valid answers:
# (4.1946, 4.1888)
# Note: The second value must be exactly 4.1888, while the first value will vary slightly due to randomness.
```

Example 2: Invalid case

```
print(monte_carlo_sphere_volume(-2.5, 500))
```

 # Expected Output:
(0.0, 0.0)

Example 3: Large radius case

```
print(monte_carlo_sphere_volume(3.5, 500000))
```

```
# Possible valid answers:
# (179.6214, 179.5944)
```

Validation Rules for Grading: For each test with valid inputs, we will run the function at least 10 times because the estimated volume is randomized. We will verify that the estimated volume changes slightly across runs to ensure true randomness, while the second element remains

perfectly fixed to the exact mathematical volume. We will also check that the estimate converges within a 5% margin of error of the actual volume.

This tool needs to be loaded in a new browser window

Load Lab 17: Math and Random in a new window

